Syntax and Semantics Lexical and Syntax Analysis.

Overview

- Introduction
- Names
- Bindings and Scopes
- Type checking
- Strong Typing
- Type Conversions
- Short-Circuit Evaluation

Introduction

- **This chapter introduces the fundamental semantic issues of variables.**
- **It begins by describing the nature of names and special words in programming language.**
- **The attributes of variables, including type, address, and value, are then discussed, including the issue of aliases.**
- **The important concepts of binding and binding times are introduced next, including the different possible binding times for variable attributes and how they define four different categories of variables.**

NAMES

- **What is name?**
- **A name is a string of characters used to identify some entity in a program. also associated with subprograms, formal parameters, and other program constructs.**
- **The term identifier is often used interchangeably with name.**

1. **Design Issue.**

- **The following are the primary design issues for names:**
- **Are names case sensitive?**
- **Are the special words of the language reserved words or keywords?**

NAMES

1.1 **Name Forms.**

- **Fortran 95+ allows up to 31 characters in its names.**

1. Python

- **Maximum Length: There is no defined maximum length for variable names in Python, but it's recommended to keep them reasonable for readability.**

2. Java

- **Maximum Length: Variable names can be very long (up to 65,535 characters), but practical usage usually keeps them shorter.**

NAMES

1.1 **Name Forms.**

3.C/C++

- **Maximum Length: The C standard specifies that identifiers (variable names) can be up to 2048 characters, while many compilers support even longer names. However, only the first 63 characters are significant for external linkage.**

NAMES

1.1 Name Forms.

- **Names in most programming languages have the same form: a letter followed by a string consisting of letters, digits, and underscore characters (_).**
- **In the C-based languages, it has to a large extent been replaced by the so-called camel notation, in which all of the words of a multiple-word name except the first are capitalized, as in myStack.**
- **In many languages, notably the C-based languages, uppercase and lowercase letters in names are distinct; that is, names in these languages are case sensitive.**

NAMES

1.1 Name Forms.

- **In many languages, notably the C-based languages, uppercase and lowercase letters in names are distinct; that is, names in these languages are case sensitive.**
- **For example, the following three names are distinct in C++: rose, ROSE, and Rose.**
- **In that sense, case sensitivity violates the design principle that language constructs that look similar should have similar meanings. But in languages whose variable names are case-sensitive, although Rose and rose look similar, there is no connection between them.**

NAMES

1.2 **Special Words.**

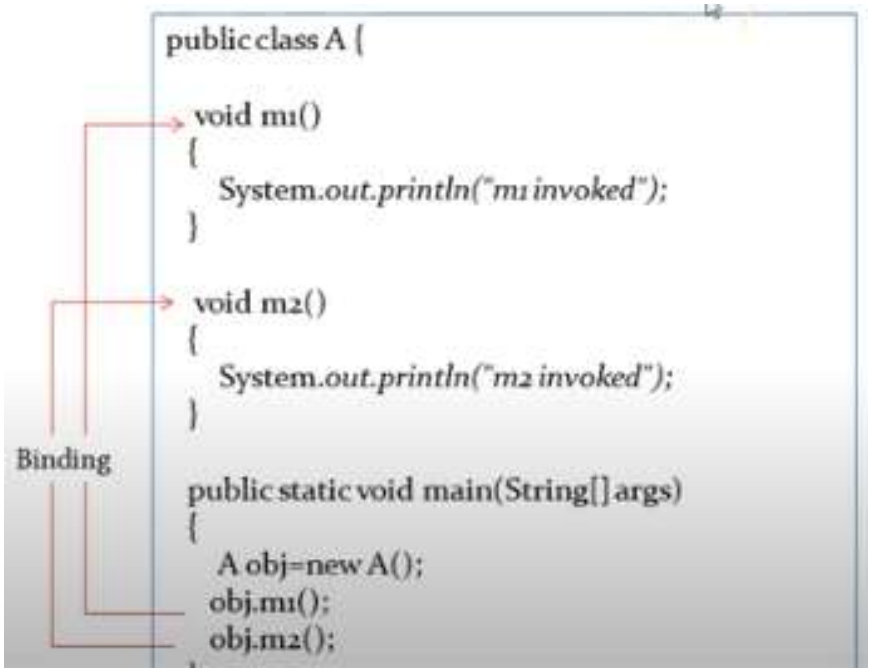
- **Special words in programming languages are used to make programs more readable by naming actions to be performed.**
- **This special word is also called as KEYWORD.**
- **There is one potential problem with reserved words. If the language includes a large number of reserved words, the user may have difficulty making up names that are not reserved.**
- **The best example of this is COBOL, which has 300 reserved words.**

Binding and Scopes

2.1.**Binding**

- What is Binding.
- **Linking between method call and method definition.**

EX:



Binding and Scopes

2.2.Types of binding.

2.2.1 Static Binding.

- Static type binding can be resolved at the time of compilation.
- This binding also called as early binding.
- Binding happens before a program actually runs.
- Example: Method Overloading.(Same name but different parameters.)

Binding and Scopes

Static Binding

```
public class Test{
  void area(float r)
  {
     float a;
     a=3.1416f*r;
     System.out.println("Area of circle :"+a);
  }
  void area(float l,float b)
  {
    float a;
    a=l*b;
    System.out.println("Area of Rectangle :"+a);
  }
  public static void main(String[] args)
  {
     Test t=new Test();
     t.area(9.5f,7.8f);
     t.area(3.4f);
```

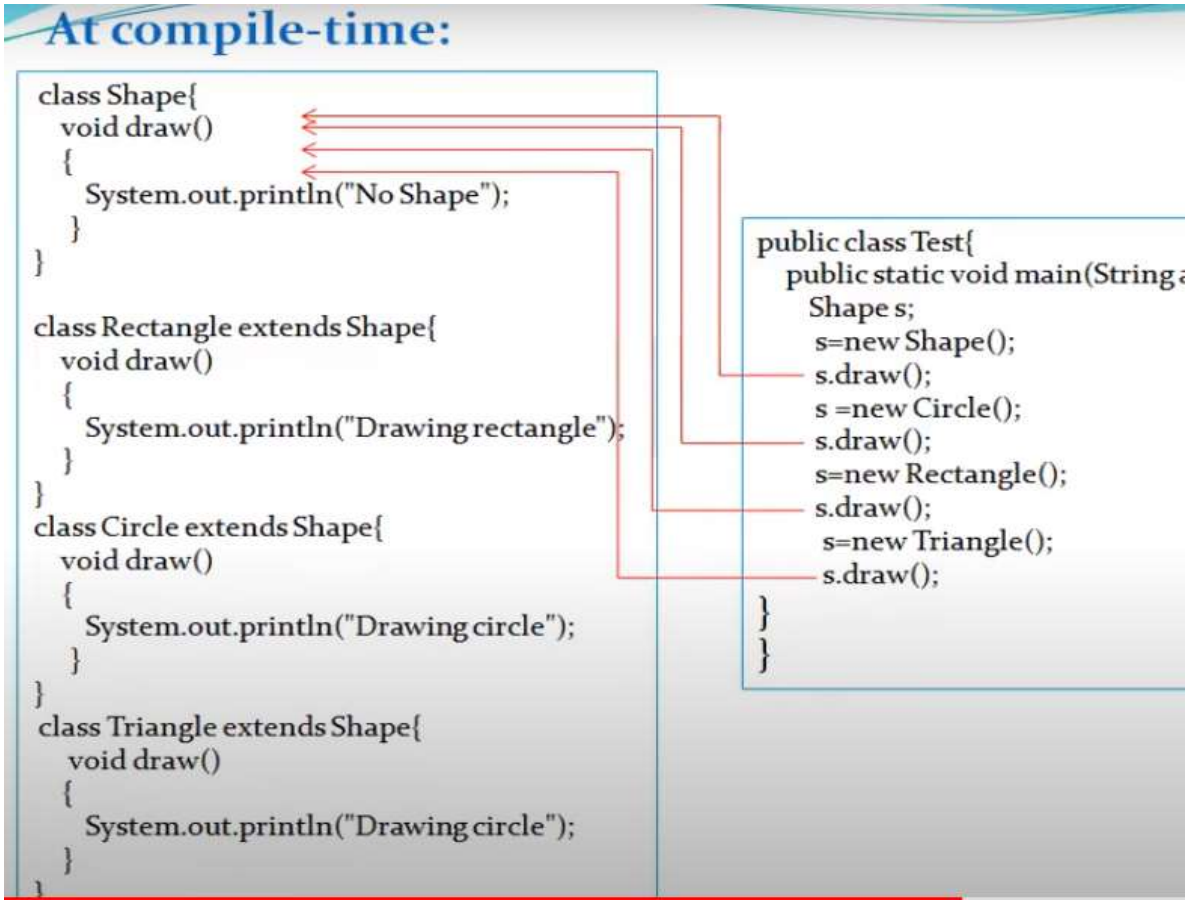
Binding and Scopes

2.2.Types of binding.

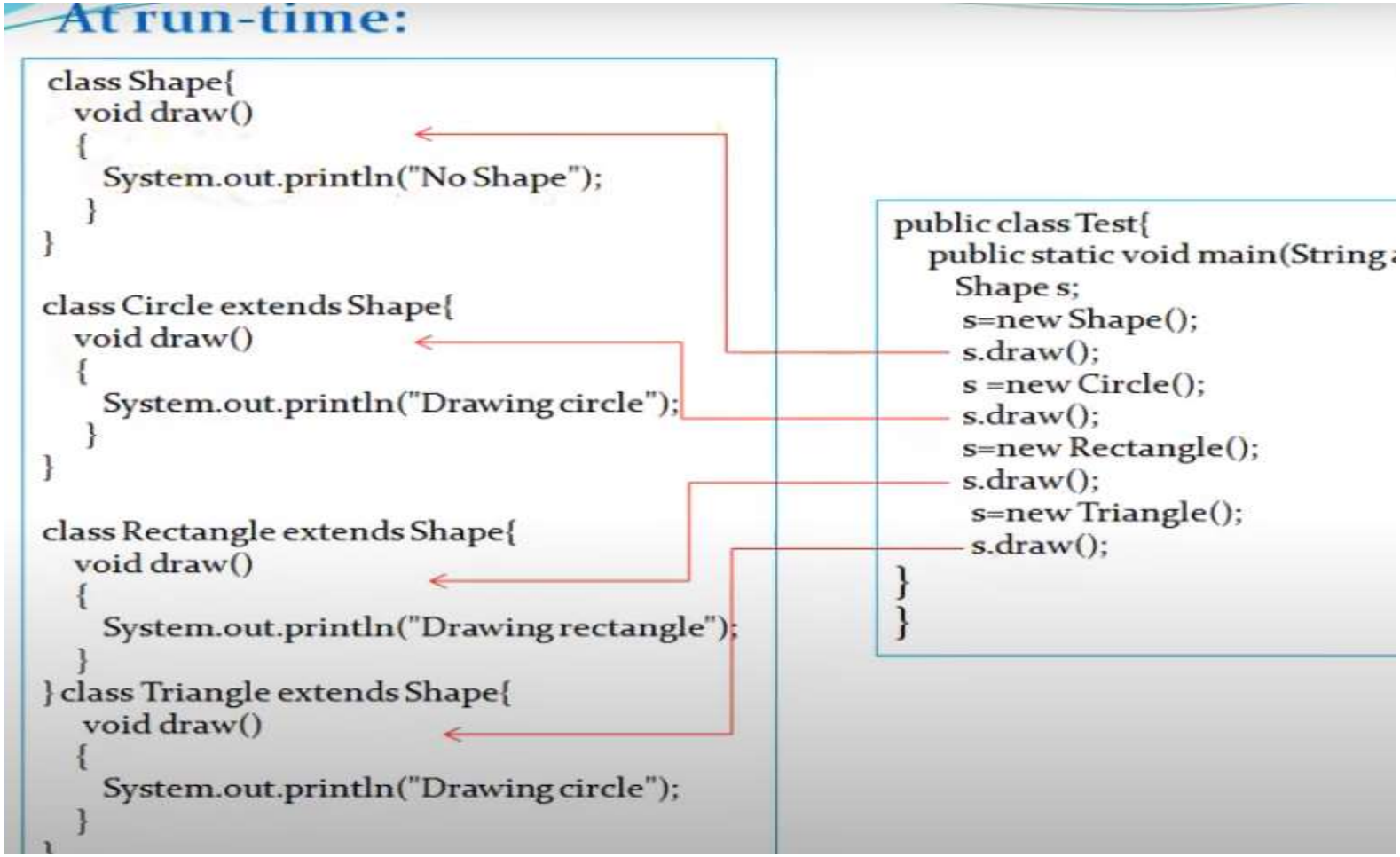
2.2.1 Dynamic Binding.

- When compiler not able to resolve the binding at compile time.
- This binding also called as late binding.
- Binding happens during runtime.
- Example:Method Overriding(same name same parameter)

Binding and Scopes



Binding and Scopes



Binding and Scopes

2.2.Storage Binding and lifetime

- The memory cell to which a variable is bound somehow must be taken from a pool of available memory. This process is called allocation.
- The process of placing a memory cell that has been unbound from a variable back into the pool of available memory. This process is called deallocation.
- The lifetime of a variable is the time during which the variable is bound to a specific memory location.
- **Lifetime of variable begins when it is bound to a specific memory cell and ends when it is unbound from that cell.**

Binding and Scopes

2.2.Storage Binding and lifetime

- Based on the lifetime of variables are categorized into four different into four different parts

1. Static Variables
2. stack-dynamic
3. explicit heap-dynamic
4. **implicit heap-dynamic.**

Binding and Scopes

2.2.1Static Variables

- Its default value is 0.
- It is stored into static memory.
- It has a function scope.
- It has a program lifetime .
- Memory is allocated to the static variable during compile time.
- **Static variable declaration execute only once during run time.**

Binding and Scopes

2.2.2Stack dynamic Variables(Auto variable)

- Its default value is garbage value.
- It is stored in stack memory during runtime.
- It has a function scope i.e. stack dynamic variable of one function is not accessible in another function.
- **It has function life time.**

Binding and Scopes

2.2.3Implicit heap dynamic Variables.

- An implicit heap dynamic variable refers to variables that are allocated on the heap without explicit allocation code by the programmer.
- Their lifetime can extend beyond the scope in which they were created.
- **In languages like Python or Java, when you create an object (e.g., new SomeObject() in Java), it is allocated on the heap, and its reference is managed automatically.**

Binding and Scopes

2.2.4 Explicit heap dynamic Variables.

- **An explicit heap dynamic variable is a variable that is explicitly allocated and deallocated by the programmer using specific memory management functions. In C, this typically involves using malloc, calloc, or realloc for allocation, and free for deallocation.**

Binding and Scopes

2.2.3 Scoping

•Scoping refers to the region in a program where a variable is accessible or can be referenced. Understanding scope is crucial for managing variable visibility, lifetime, and memory usage in programming. Here's a breakdown of the different types of scoping.

2.2.3.1 Type of Scope.

1. Global Scope
2. Local Scope

Binding and Scopes

2.2.3.1 Type of Scope.

1. **Global Scope**

- **Variables defined in the global scope can be accessed from any part of the program.**
- **They exist for the duration of the program's execution.**

Binding and Scopes

2.2.3.1 Type of Scope.

1. **Local Scope**

- **Variables defined within a function or block are local to that function/block and cannot be accessed outside of it.**
- **They exist only during the execution of that function/block.**

Binding and Scopes

2.2.3.2 Rules of Scope.

1. **Static Scope**

- **Referencing environment of a statement in a statistically scope language is the collection of variable which are locally declared and all other ancestor variable which are visible in the statement.**

Binding and Scopes

2.2.3.2 Rules of Scope.

1. **Dynamic Scope**

- **Referencing environment of a statement in a dynamically scope language is the collection of all variables which are locally declared and all other active subprogram variable which are visible in the statement.**